Circularo CLI Search Utility Requirements

Purpose

This specification defines a hiring exercise for a developer who will build a CLI application for working with Circularo data through the REST API. The goal is not to reproduce an exact implementation from memory, but to see how effectively the developer can discover requirements, structure the repository, design the application, and execute the work using an agentic development environment.

The finished result should be a practical CLI utility capable of searching Circularo resources, producing deterministic output, and serving as a solid foundation for future automation work.

Evaluation Focus

The exercise should measure the candidate on the following dimensions:

- how quickly they orient themselves in the repo and external documentation
- how clearly they structure the codebase and supporting docs
- how well they use an agentic workflow to plan, implement, verify, and refine the work
- how effectively they map Circularo API behavior into a usable command-line product
- how well they preserve deterministic, machine-friendly output
- how responsibly they handle unclear API behavior, missing information, and open questions

The emphasis is on judgment and execution quality, not on producing the largest possible feature set.

Expected Outcome

The candidate should deliver a working CLI project that can search Circularo resources and return results in predictable formats suitable for humans and downstream automation. The implementation should be organized so that another developer or agent can understand the repository quickly and continue extending it without needing to reverse-engineer the design.

The project should feel intentionally built rather than assembled from a single script.

Scope

The CLI should support the core search use case for Circularo data. It should be able to query at least the main resource types relevant to the workspace and expose a clean command model that can grow over time.

The implementation should cover:

- a coherent command-line interface
- API client integration for Circularo
- search request construction and translation
- deterministic result rendering
- clear error handling
- documentation for usage and maintenance
- tests that protect the main behavior

The exercise may be scoped to search-focused functionality only, unless the candidate explicitly proposes and justifies a broader scope.

Repository Expectations

The candidate should create a repository layout that makes the system easy to navigate. The structure should separate concerns cleanly enough that CLI parsing, API access, query handling, and output formatting do not bleed into one another.

The repo should also include the following:

- a top-level README that explains the product and how to run it
- a small set of supporting docs for design decisions and usage
- a test suite that covers the main command flows
- a clear place for unresolved questions and future work
- configuration that supports repeatable local execution

The final layout should be easy for an agentic tool and a human reviewer to inspect without extra explanation.

Design Expectations

The implementation should be modular and deliberate. The candidate should show that they can identify stable boundaries in the system and keep those boundaries intact while iterating.

The design should include, at minimum:

- a thin CLI layer
- a dedicated API layer
- a search or query translation layer
- an output formatting layer
- a configuration layer for environment and credentials

The candidate should be able to explain why the code is divided that way and what tradeoffs they chose.

Agentic Workflow Expectations

The exercise is specifically intended to test how well the candidate works with agentic AI tools. The candidate may use any reasonable agentic environment, including opencode, Claude, Codex, Gemini CLI, or a comparable workflow, but they should be able to demonstrate disciplined use of that environment rather than simply prompting their way through the task.

The expected workflow is:

- read the local project docs before changing code
- understand the existing repository conventions before making assumptions
- plan the work in small steps
- implement incrementally
- verify the result after each meaningful change
- keep notes on unresolved questions and design decisions
- avoid creating unnecessary complexity when a simpler structure is sufficient

The candidate should show that they can use agentic tools to accelerate work while still retaining control over architecture, correctness, and maintainability.

Circularo API Expectations

The candidate should demonstrate that they can work effectively with the Circularo API and its documentation. That includes understanding how to locate relevant endpoints, how to interpret the shape of returned data, and how to decide which API behaviors are stable enough to wrap in a CLI contract.

For this exercise, the only authoritative sources for Circularo API integration are:

- https://developers.circularo.com/
- https://www.circularo.com/api/full/26.0.0/
- https://www.circularo.com/api/integrators/26.0.0/

The implementation should reflect a sound reading of the API rather than a guess-driven integration. If the API surface is ambiguous, the candidate should surface that ambiguity clearly and choose a reasonable approach that can be defended in review.

The candidate should pay attention to:

- authentication and credential handling
- the difference between resource types
- query and filter construction
- pagination or large-result handling where relevant
- field naming and output stability
- error propagation from the API to the CLI

Functional Requirements

The CLI should provide a small, coherent command set for searching Circularo data. The exact command names are open to design, but the behavior should be easy to discover, consistent across resource types, and stable enough to support automation.

At minimum, the utility should:

- search Circularo resources by user-supplied criteria
- support structured and unstructured query input where appropriate
- return results in machine-readable and human-readable formats
- preserve raw response details when useful for downstream processing
- handle large result sets without becoming interactive or unpredictable

The tool should remain deterministic. Repeated runs with the same inputs should produce equivalent outputs unless the underlying data has changed.

Output Requirements

The CLI should support output modes that are useful both for automation and for manual inspection. The candidate should make deliberate choices about how much raw data to preserve and how much to project into summaries.

The output should:

- remain stable in field naming and ordering where possible
- support at least one structured machine-readable format
- support at least one tabular or human-readable format
- include enough metadata to trace what was requested and what was returned

The implementation should avoid ad hoc formatting that makes later automation difficult.

Error Handling Requirements

The tool should fail clearly and predictably when something goes wrong. The candidate should not hide API failures or collapse them into vague CLI messages.

The CLI should distinguish between:

- invalid or missing credentials
- authorization failures
- malformed input
- empty result sets
- upstream API errors
- local configuration problems

Error messages should help a user or agent understand what failed and where to look next.

Documentation Requirements

The candidate should provide documentation that makes the project usable without a live walkthrough. The docs should explain the purpose of the tool, the main command model, how to configure it, and how the repository is organized.

The documentation should also capture:

- the candidate's design choices
- any limitations they intentionally accepted
- any open questions or assumptions that remain
- any API behavior that was unclear or tenant-specific

The quality of the documentation is part of the evaluation.

Testing Requirements

The candidate should include tests that prove the CLI behaves as expected for the main workflows. Tests should be targeted and deterministic rather than exhaustive for their own sake.

The test suite should cover:

- command parsing or dispatch
- request construction
- output formatting
- error handling
- at least one end-to-end happy path, if practical

The testing approach should reflect good judgment about what needs to be mocked and what should be exercised directly.

Delivery Expectations

The candidate should be able to explain the repository structure, the design decisions, the agentic workflow they used, and the way they mapped Circularo API behavior into the CLI.

The final delivery should include:

- the working CLI project
- documentation of the implemented behavior
- tests
- a concise list of assumptions and open questions
- a short summary of lessons learned
- a candid note on any struggles, gaps, or unresolved risks
- recommendations for how the project could be improved in a follow-up iteration

The candidate should also stage, commit, and push their work to a private repository so that progress, decision history, and delivery quality can be reviewed over time.

Evaluation Criteria

The primary evaluation criterion is the quality of the engineering approach: how well the candidate blends human judgment with agentic tooling to deliver well documented, maintainable code in a well structured repository that preserves key decisions and project history.

The most important factors are:

- repository structure and maintainability
- documentation quality and decision tracking
- clarity of architecture and separation of concerns
- correctness of the Circularo API integration
- traceability of assumptions, gaps, and design choices
- quality of tests and verification

Code quality, architectural clarity, and API correctness matter more than technology choice, framework choice, or raw speed of execution by themselves.

We will also assess delivery speed in a practical sense: how quickly the candidate produces tangible results, while still ending with a well engineered and well structured repository and codebase.

Because progress tracking is part of the evaluation, the candidate should commit early, commit often enough to make their work reviewable, and push those commits to a private repository during the exercise.

Non-Goals

This exercise is not primarily about maximizing feature count. It is not a request for a large suite of unrelated commands, a broad admin console, or a highly opinionated product layer that hides API behavior.

The important part is whether the candidate can build a useful, maintainable CLI and work effectively with agentic tooling and Circularo documentation.

Open Questions

These are not just questions for the evaluator. They are also the kinds of decisions the candidate is expected to think through, resolve, and record clearly as part of the exercise. The repository should preserve those decisions, assumptions, and tradeoffs so the review can trace how the final design was reached.

Scope And Product Boundary

- The candidate should decide whether the project focuses only on search or also includes adjacent workflows such as export, reporting, or evidence packaging, and record that scope decision clearly.
- The candidate should decide whether the CLI targets one clearly defined Circularo resource area or demonstrates breadth across multiple resource types, and record the product boundary clearly.

Agentic Workflow

- The candidate should decide which agentic environment to use, if any, and explain why that workflow was selected.
- The candidate should decide whether to use one environment consistently or switch between tools as needed, and record the rationale.
- The candidate should document prompts, planning, decision history, and other workflow notes when they are useful to understanding how the delivery was produced.

Circularo API Boundaries

- The candidate should decide which auth mechanism is the default integration path and record that assumption.
- The candidate should decide which API behaviors are stable enough to rely on and which ones should be treated as ambiguous or tenant-specific.
- The candidate should decide which resource types or endpoints are in scope or out of scope, and make that boundary explicit in the repository.
- The candidate should decide whether the implementation optimizes for live API integration, offline mockability, or both, and explain the tradeoff.

Implementation Constraints

- The candidate should decide the runtime, dependency style, and packaging approach, and record any constraints they adopted.
- The candidate should decide whether the CLI supports one output format first or multiple formats from the beginning, and document the staged approach if one is used.
- The candidate should decide the repository layout and coding conventions, and explain how those choices support maintainability and future work.

Review Criteria

- What matters more in scoring: code quality, architectural clarity, API correctness, or speed of execution?
- Should the evaluation reward cautious handling of uncertainty, or a stronger bias toward making decisions quickly?
- Do you want a strict pass/fail checklist, or a more open-ended assessment rubric?

These review questions are for the evaluator, but the candidate should still show awareness of them through the decisions they record and the structure of their delivery.

CLI Surface Inspiration

This section is intentionally high level. It is meant to suggest the kind of command surface we expect, not to prescribe the exact implementation.

The CLI should feel like a small, coherent utility with a discoverable help tree and clearly separated areas for the main resource types and reporting workflows.

The surface could look roughly like this:

```
circularo-cli --help
circularo-cli users --help
circularo-cli documents --help
circularo-cli reports --help
circularo-cli transactions --help
```

The candidate should make sensible design choices for command names, flags, and subcommands, but the tool should support at least:

- user search

- document search
- report-oriented workflows

Concrete examples may include:

```
circularo-cli users --query "mary griffin"
circularo-cli documents --query "completed invoices"
circularo-cli reports --help
circularo-cli documents --query "title:Invoice*" --format json
circularo-cli users --query "josef.neumann@circularo.com" --format md
circularo-cli transactions  --state completed
```

The exact flag names and command structure may differ, but the final experience should be easy to discover, consistent to use, and suitable for later automation.